

目录

二维计算几何基础	1
整数点、点积和叉积	1
跨立实验与线段相交	2
多边形面积与点包含	3
Pick 定理与格点计数	4
凸包与旋转卡壳	5
Andrew 凸包	5
凸包直径	6
浮点几何、直线与圆	6
半平面交	9
凸多边形 Minkowski 和	10

二维计算几何基础

计算几何最容易出错的地方不是公式，而是谓词是否精确。当输入坐标是整数时，方向、共线、跨立和点在多边形内等离散判断应优先使用整数叉积；只有交点坐标、距离和圆等必须产生实数的操作才使用浮点数。

下面约定整数坐标和差的绝对值不超过 10^9 。叉积使用 `__int128_t`，因此不会发生 64 位乘法溢出。

整数点、点积和叉积

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

namespace geometry {

using i64 = long long;
using i128 = __int128_t;

struct IPoint {
    i64 x = 0;
    i64 y = 0;

    IPoint operator+(const IPoint& other) const {
        return {x + other.x, y + other.y};
    }
    IPoint operator-(const IPoint& other) const {
        return {x - other.x, y - other.y};
    }
    bool operator==(const IPoint& other) const {
        return x == other.x && y == other.y;
    }
    bool operator<(const IPoint& other) const {
        return tie(x, y) < tie(other.x, other.y);
    }
};
```

```

i128 dot(const IPoint& lhs, const IPoint& rhs) {
    return (i128)lhs.x * rhs.x + (i128)lhs.y * rhs.y;
}

i128 cross(const IPoint& lhs, const IPoint& rhs) {
    return (i128)lhs.x * rhs.y - (i128)lhs.y * rhs.x;
}

i128 cross(const IPoint& origin, const IPoint& lhs, const IPoint& rhs) {
    return cross(lhs - origin, rhs - origin);
}

i128 norm2(const IPoint& point) {
    return dot(point, point);
}

int sign(i128 value) {
    return (value > 0) - (value < 0);
}

} // namespace geometry

```

$\text{cross}(a, b, c)$ 的符号表示从向量 $\overrightarrow{ab}$ 转到 $\overrightarrow{ac}$ 的方向：正数为逆时针，负数为顺时针，零为共线。

跨立实验与线段相交

线段 ab 与直线 cd 的跨立实验是比较 $\text{cross}(c, d, a)$ 与 $\text{cross}(c, d, b)$ 的符号。两条线段严格相交，当且仅当它们分别严格跨立对方所在直线。

只用严格跨立会漏掉端点接触和共线重叠，因此完整实现必须单独处理叉积为零的情形：

```

namespace geometry {

bool on_segment(const IPoint& point, const IPoint& lhs, const IPoint& rhs) {
    return cross(lhs, rhs, point) == 0 &&
        min(lhs.x, rhs.x) <= point.x && point.x <= max(lhs.x, rhs.x) &&
        min(lhs.y, rhs.y) <= point.y && point.y <= max(lhs.y, rhs.y);
}

enum class SegmentRelation {
    disjoint,           // 没有公共点
    touch,              // 恰有一个公共点，但不是两线段内部严格相交
    proper_intersection, // 两线段在各自内部严格相交
    overlap             // 共线，且交集是一段非零长度线段
};

SegmentRelation segment_relation(IPoint a, IPoint b, IPoint c, IPoint d) {
    int ab_c = sign(cross(a, b, c));
    int ab_d = sign(cross(a, b, d));
    int cd_a = sign(cross(c, d, a));
    int cd_b = sign(cross(c, d, b));

    if (ab_c == 0 && ab_d == 0) {
        if (b < a) swap(a, b);
        if (d < c) swap(c, d);
    }
}

```

```

    IPoint left = max(a, c);
    IPoint right = min(b, d);
    if (right < left) return SegmentRelation::disjoint;
    if (left == right) return SegmentRelation::touch;
    return SegmentRelation::overlap;
}

if (ab_c * ab_d < 0 && cd_a * cd_b < 0) {
    return SegmentRelation::proper_intersection;
}
if ((ab_c == 0 && on_segment(c, a, b)) ||
    (ab_d == 0 && on_segment(d, a, b)) ||
    (cd_a == 0 && on_segment(a, c, d)) ||
    (cd_b == 0 && on_segment(b, c, d))) {
    return SegmentRelation::touch;
}
return SegmentRelation::disjoint;
}

} // namespace geometry

```

复杂度为 $O(1)$ 。如果题目只问“是否有公共点”，返回值不是 `disjoint` 即可。

多边形面积与点包含

`point_in_polygon` 适用于任意简单多边形，顶点可顺时针或逆时针给出，返回 `outside` / `boundary` / `inside`。射线经过顶点时必须使用半开区间判断，否则同一个顶点可能被统计两次。

```

namespace geometry {

i128 polygon_area_twice(const vector<IPoint>& polygon) {
    i128 area = 0;
    int size = (int)polygon.size();
    for (int i = 0; i < size; ++i) {
        area += cross(polygon[i], polygon[(i + 1) % size]);
    }
    return area;
}

enum class PointLocation {
    outside,    // 点在多边形外部
    boundary,   // 点在多边形边界上
    inside      // 点在多边形内部
};

PointLocation point_in_polygon(const vector<IPoint>& polygon, const IPoint& point) {
    bool inside = false;
    int size = (int)polygon.size();
    for (int i = 0; i < size; ++i) {
        IPoint lhs = polygon[i];
        IPoint rhs = polygon[(i + 1) % size];
        if (on_segment(point, lhs, rhs)) return PointLocation::boundary;

        bool crosses_height = (lhs.y > point.y) != (rhs.y > point.y);
    }
    return inside ? PointLocation::inside : PointLocation::outside;
}

```

```

        if (!crosses_height) continue;
        i128 direction = cross(lhs, rhs, point);
        if ((direction > 0) == (rhs.y > lhs.y)) inside = !inside;
    }
    return inside ? PointLocation::inside : PointLocation::outside;
}

} // namespace geometry

```

面积的实际值是 $\text{abs}(\text{polygon_area_twice}(\text{polygon})) / 2$ 。点包含单次查询复杂度为 $O(n)$ 。

Pick 定理与格点计数

对顶点都在整点上、没有自交且没有洞的简单多边形，设面积为 A 、内部格点数为 I 、边界格点数为 B ，则

$$A = I + \frac{B}{2} - 1.$$

使用二倍面积 $A_2 = 2A$ 时无需浮点数：

$$I = \frac{A_2 - B + 2}{2}.$$

整数线段两端坐标差为 (dx, dy) 时，线段被分成 $\gcd(|dx|, |dy|)$ 段最短格点线段。沿多边形各边累加该 $\gcd$ ，端点恰好各计一次，因此

$$B = \sum_{i=0}^{n-1} \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|).$$

```

namespace geometry {

i128 boundary_lattice_points(const vector<IPoint>& polygon) {
    i128 boundary = 0;
    int size = (int)polygon.size();
    for (int i = 0; i < size; ++i) {
        const IPoint& lhs = polygon[i];
        const IPoint& rhs = polygon[(i + 1) % size];
        i64 dx = labs(rhs.x - lhs.x);
        i64 dy = labs(rhs.y - lhs.y);
        boundary += gcd(dx, dy);
    }
    return boundary;
}

i128 interior_lattice_points(const vector<IPoint>& polygon) {
    i128 area_twice = polygon_area_twice(polygon);
    if (area_twice < 0) area_twice = -area_twice;
    i128 boundary = boundary_lattice_points(polygon);
    i128 numerator = area_twice - boundary + 2;
    assert(polygon.size() >= 3 && area_twice > 0);
    assert(numerator >= 0 && numerator % 2 == 0);
    return numerator / 2;
}

```

```
} // namespace geometry
```

单条闭线段上的格点数是 $\gcd(\text{abs}(\text{dx}), \text{abs}(\text{dy})) + 1$ ，但按多边形边累加时不能再加 1，否则每个顶点会被重复计算。Pick 定理不能直接用于自交多边形；有 h 个洞时应改为 $A = I + B/2 - 1 + h$ 。

凸包与旋转卡壳

以下函数依赖 01-geometry.md 中的 IPoint、cross、norm2 和 i128。输入点允许重复。默认删除凸包边上的中间共线点；keep_collinear=true 时保留边界上的共线点。

Andrew 凸包

```
namespace geometry {

vector<IPoint> convex_hull(vector<IPoint> points, bool keep_collinear = false) {
    sort(points.begin(), points.end());
    points.erase(unique(points.begin(), points.end()), points.end());
    if (points.size() <= 1) return points;

    if (keep_collinear) {
        bool all_collinear = true;
        for (int i = 2; i < (int)points.size(); ++i) {
            if (cross(points[0], points[1], points[i]) != 0) {
                all_collinear = false;
                break;
            }
        }
        if (all_collinear) return points;
    }

    vector<IPoint> hull;
    auto should_pop = [&](const IPoint& next) {
        i128 turn = cross(hull[hull.size() - 2], hull.back(), next);
        return keep_collinear ? turn < 0 : turn <= 0;
    };

    for (const IPoint& point : points) {
        while (hull.size() >= 2 && should_pop(point)) hull.pop_back();
        hull.push_back(point);
    }
    int lower_size = (int)hull.size();
    for (int i = (int)points.size() - 2; i >= 0; --i) {
        while ((int)hull.size() > lower_size && should_pop(points[i])) {
            hull.pop_back();
        }
        hull.push_back(points[i]);
    }
    hull.pop_back();
    return hull;
}
```

```
} // namespace geometry
```

输出不重复首点，并按逆时针排列，从字典序最小点开始。复杂度由排序主导，为 $O(n \log n)$ 。

凸包直径

凸包直径是最远点对距离。输入必须是 `keep_collinear=false` 得到的逆时针凸包。

```
namespace geometry {

i128 convex_diameter_squared(const vector<IPoint>& hull) {
    int size = (int)hull.size();
    if (size <= 1) return 0;
    if (size == 2) return norm2(hull[1] - hull[0]);

    i128 answer = 0;
    int opposite = 1;
    for (int i = 0; i < size; ++i) {
        int next_i = (i + 1) % size;
        while (true) {
            int next_opposite = (opposite + 1) % size;
            i128 current_area = cross(hull[next_i] - hull[i],
                                     hull[opposite] - hull[i]);
            i128 next_area = cross(hull[next_i] - hull[i],
                                   hull[next_opposite] - hull[i]);
            if (next_area <= current_area) break;
            opposite = next_opposite;
        }
        answer = max(answer, norm2(hull[i] - hull[opposite]));
        answer = max(answer, norm2(hull[next_i] - hull[opposite]));
    }
    return answer;
}

} // namespace geometry
```

复杂度为 $O(h)$ ，其中 h 是凸包点数。返回距离平方，避免不必要的开方和浮点误差。

浮点几何、直线与圆

只有必须构造非整数坐标时才进入浮点层。不要直接用 `==` 比较 `long double`，并且不要把 `eps` 当作任意数量级下的万能容差；坐标极大时应按题目范围调整。

```
#include <bits/stdc++.h>
using namespace std;

namespace geometry_float {

using ld = long double;
constexpr ld eps = 1e-12L;
const ld pi = acosl(-1.0L);
```

```

int sign(ld value) {
    return (value > eps) - (value < -eps);
}

struct Point {
    ld x = 0;
    ld y = 0;

    Point operator+(const Point& other) const {
        return {x + other.x, y + other.y};
    }
    Point operator-(const Point& other) const {
        return {x - other.x, y - other.y};
    }
    Point operator*(ld factor) const {
        return {x * factor, y * factor};
    }
    Point operator/(ld factor) const {
        return {x / factor, y / factor};
    }
};

ld dot(const Point& lhs, const Point& rhs) {
    return lhs.x * rhs.x + lhs.y * rhs.y;
}

ld cross(const Point& lhs, const Point& rhs) {
    return lhs.x * rhs.y - lhs.y * rhs.x;
}

ld norm2(const Point& point) {
    return dot(point, point);
}

ld length(const Point& point) {
    return sqrtl(norm2(point));
}

Point rotate90(const Point& point) {
    return {-point.y, point.x};
}

struct Line {
    Point point;
    Point direction;
};

optional<Point> line_intersection(const Line& lhs, const Line& rhs) {
    ld denominator = cross(lhs.direction, rhs.direction);
    if (sign(denominator) == 0) return nullopt;
    ld ratio = cross(rhs.point - lhs.point, rhs.direction) / denominator;
    return lhs.point + lhs.direction * ratio;
}

```

```

Point projection(const Point& point, const Line& line) {
    return line.point + line.direction *
        (dot(point - line.point, line.direction) / norm2(line.direction));
}

ld distance_to_line(const Point& point, const Line& line) {
    return fabs1(cross(line.direction, point - line.point)) /
        length(line.direction);
}

struct Circle {
    Point center;
    ld radius = 0;
};

vector<Point> circle_intersections(const Circle& lhs, const Circle& rhs) {
    Point difference = rhs.center - lhs.center;
    ld center_distance = length(difference);
    if (center_distance <= eps) return {};
    if (center_distance > lhs.radius + rhs.radius + eps) return {};
    if (center_distance < fabs1(lhs.radius - rhs.radius) - eps) return {};

    ld along = (lhs.radius * lhs.radius - rhs.radius * rhs.radius +
        center_distance * center_distance) / (2 * center_distance);
    ld height_squared = lhs.radius * lhs.radius - along * along;
    if (height_squared < -eps) return {};
    height_squared = max<ld>(0, height_squared);

    Point unit = difference / center_distance;
    Point base = lhs.center + unit * along;
    Point offset = rotate90(unit) * sqrt1(height_squared);
    if (height_squared <= eps) return {base};
    return {base + offset, base - offset};
}

vector<Point> line_circle_intersections(const Line& line, const Circle& circle) {
    Point foot = projection(circle.center, line);
    ld distance_squared = norm2(foot - circle.center);
    ld height_squared = circle.radius * circle.radius - distance_squared;
    if (height_squared < -eps) return {};
    height_squared = max<ld>(0, height_squared);
    if (height_squared <= eps) return {foot};
    Point offset = line.direction / length(line.direction) * sqrt1(height_squared);
    return {foot + offset, foot - offset};
}

} // namespace geometry_float

```

重合圆有无穷多个交点，上面的 `circle_intersections` 也返回空数组；若题目需要区分“无交点”和“重合”，应在调用前单独判断圆心距离与半径差。传给投影、点到直线距离和直线圆交点的方向向量必须非零。

半平面交

每条有向直线保留其左侧半平面。下面实现要求最终交集有界；常见做法是额外加入一个足够大的逆时针矩形。空数组既可能表示空集，也可能表示无界区域。

依赖 03-floating-geometry.md 中的 Point、Line、cross、dot、length、sign、eps 和 line_intersection。

```
namespace geometry_float {

bool outside(const Line& line, const Point& point) {
    return cross(line.direction, point - line.point) < -eps;
}

vector<Point> half_plane_intersection(vector<Line> lines) {
    auto angle = [](const Line& line) {
        return atan2l(line.direction.y, line.direction.x);
    };
    sort(lines.begin(), lines.end(), [&](const Line& lhs, const Line& rhs) {
        ld lhs_angle = angle(lhs);
        ld rhs_angle = angle(rhs);
        if (lhs_angle != rhs_angle) return lhs_angle < rhs_angle;
        return cross(lhs.direction, rhs.point - lhs.point) > 0;
    });

    vector<Line> unique_lines;
    for (const Line& line : lines) {
        if (length(line.direction) <= eps) continue;
        if (!unique_lines.empty() &&
            sign(cross(unique_lines.back().direction, line.direction)) == 0 &&
            dot(unique_lines.back().direction, line.direction) > 0) {
            if (cross(unique_lines.back().direction,
                line.point - unique_lines.back().point) > eps) {
                unique_lines.back() = line;
            }
        } else {
            unique_lines.push_back(line);
        }
    }

    deque<Line> active;
    deque<Point> intersections;
    for (const Line& line : unique_lines) {
        while (!intersections.empty() && outside(line, intersections.back())) {
            intersections.pop_back();
            active.pop_back();
        }
        while (!intersections.empty() && outside(line, intersections.front())) {
            intersections.pop_front();
            active.pop_front();
        }
        if (!active.empty()) {
            optional<Point> point = line_intersection(active.back(), line);
            if (!point.has_value()) return {};
            intersections.push_back(*point);
        }
    }
}
```

```

        active.push_back(line);
    }

    while (!intersections.empty() && outside(active.front(), intersections.back())) {
        intersections.pop_back();
        active.pop_back();
    }
    while (!intersections.empty() && outside(active.back(), intersections.front())) {
        intersections.pop_front();
        active.pop_front();
    }
    if (active.size() < 3) return {};

    optional<Point> closing = line_intersection(active.back(), active.front());
    if (!closing.has_value()) return {};
    intersections.push_back(*closing);
    return {intersections.begin(), intersections.end()};
}

} // namespace geometry_float

```

排序复杂度为 $O(n \log n)$ ，双端队列扫描为 $O(n)$ 。平行线去重时保留更靠左、约束更强的那一条。

凸多边形 Minkowski 和

两个点集的 Minkowski 和定义为

$$A + B = \{a + b : a \in A, b \in B\}.$$

对严格凸多边形，其边向量分别按极角有序；合并两组边向量即可在线性时间得到和多边形。代码依赖 01-geometry.md 的整数点与 02-convex-hull.md。

```

namespace geometry {

vector<IPoint> rotate_convex_start(vector<IPoint> polygon) {
    if (polygon.empty()) return polygon;
    int start = 0;
    for (int index = 1; index < (int)polygon.size(); ++index) {
        if (tie(polygon[index].y, polygon[index].x) <
            tie(polygon[start].y, polygon[start].x)) start = index;
    }
    rotate(polygon.begin(), polygon.begin() + start, polygon.end());
    return polygon;
}

vector<IPoint> minkowski_sum(vector<IPoint> left,
                           vector<IPoint> right) {
    if (left.empty() || right.empty()) return {};

    // 输入应是 convex_hull(..., false) 的输出；退化到点或线段时直接做常数枚枚举。
    if (left.size() <= 2 || right.size() <= 2) {
        vector<IPoint> sums;
    }
}

```

```

    for (const IPoint& lhs : left) {
        for (const IPoint& rhs : right) sums.push_back(lhs + rhs);
    }
    return convex_hull(move(sums));
}

left = rotate_convex_start(move(left));
right = rotate_convex_start(move(right));
vector<IPoint> left_edges(left.size());
vector<IPoint> right_edges(right.size());
for (int index = 0; index < (int)left.size(); ++index) {
    left_edges[index] = left[(index + 1) % left.size()] - left[index];
}
for (int index = 0; index < (int)right.size(); ++index) {
    right_edges[index] = right[(index + 1) % right.size()] - right[index];
}

vector<IPoint> result;
IPoint current = left[0] + right[0];
result.push_back(current);
int left_index = 0;
int right_index = 0;
while (left_index < (int)left_edges.size() ||
        right_index < (int)right_edges.size()) {
    IPoint step;
    if (right_index == (int)right_edges.size()) {
        step = left_edges[left_index++];
    } else if (left_index == (int)left_edges.size()) {
        step = right_edges[right_index++];
    } else {
        int direction = cross(left_edges[left_index],
                               right_edges[right_index]);
        if (direction > 0) {
            step = left_edges[left_index++];
        } else if (direction < 0) {
            step = right_edges[right_index++];
        } else {
            step = left_edges[left_index++] + right_edges[right_index++];
        }
    }
    current = current + step;
    result.push_back(current);
}
assert(result.front() == result.back());
result.pop_back();
return result;
}

} // namespace geometry

```

前提是两个输入均为 `convex_hull(points, false)` 产生的严格凸包：不重复首点、逆时针、无共线中间点。主体复杂度 $O(|A| + |B|)$ ；点和线段的退化分支只枚举至多四个点。

若原始输入还不是凸包，先分别求凸包，总复杂度由排序变成 $O(n \log n + m \log m)$ 。叉积与点加法的范围

也必须满足 $i128/i64$ 的约束；Minkowski 和坐标是两边坐标之和。